

SKILL 23

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    printf("=== Task 23: Pipeline and Performance Test ===\n");

    clock_t start = clock();

    printf("\n=== Large Pipeline Test ===\n");

    int result = system("seq 1 100 | grep 5 | sort -n | uniq | wc -l");

    if (result == 0)
        printf("Large pipeline executed successfully\n");
    else
        printf("Pipeline execution failed\n");

    printf("\n=== Multiple Jobs Test ===\n");

    pid_t pid1 = fork();

    if (pid1 == 0) {
        printf("Job 1 started, PID = %d\n", getpid());
        sleep(2);
        printf("Job 1 completed\n");
        exit(0);
    }

    pid_t pid2 = fork();

    if (pid2 == 0) {
        printf("Job 2 started, PID = %d\n", getpid());
        sleep(3);
        printf("Job 2 completed\n");
        exit(0);
    }

    pid_t pid3 = fork();
```

```
if (pid3 == 0) {
    printf("Job 3 started, PID = %d\n", getpid());
    sleep(1);
    printf("Job 3 completed\n");
    exit(0);
}

printf("\n=== Resource Monitoring ===\n");

system("ps -o pid,ppid,%cpu,%mem,cmd -p $$");

printf("\n=== Waiting for Jobs ===\n");

int status;

waitpid(pid1, &status, 0);

if (WIFEXITED(status))
    printf("Job 1 exit status: %d\n", WEXITSTATUS(status));
else
    printf("Job 1 failed\n");

waitpid(pid2, &status, 0);

if (WIFEXITED(status))
    printf("Job 2 exit status: %d\n", WEXITSTATUS(status));
else
    printf("Job 2 failed\n");

waitpid(pid3, &status, 0);

if (WIFEXITED(status))
    printf("Job 3 exit status: %d\n", WEXITSTATUS(status));
else
    printf("Job 3 failed\n");

printf("\n=== Stability Test ===\n");
printf("All jobs completed successfully\n");

printf("\n=== Failure Detection Test ===\n");

result = system("wrong_command_xyz > /dev/null 2>&1");

if (result != 0)
    printf("Failure detected successfully\n");
```

```

    clock_t end = clock();

    double time_taken =
        ((double)(end - start)) / CLOCKS_PER_SEC;

    printf("\n=== Performance Analysis ===\n");
    printf("CPU execution time: %.4f seconds\n", time_taken);

    printf("\n=== Task 23 Complete ===\n");

    return 0;
}

```

Output:

```

=== Task 23: Pipeline and Performance Test ===

=== Large Pipeline Test ===
19
Large pipeline executed successfully

=== Multiple Jobs Test ===
Job 1 started, PID = 897
Job 2 started, PID = 898

=== Resource Monitoring ===
Job 3 started, PID = 899
  PID    PPID %CPU %MEM CMD
  900     889  0.0  0.0 sh -c -- ps -o pid,ppid,%cpu,%mem,cmd -p $$

=== Waiting for Jobs ===
Job 3 completed
Job 1 completed
Job 1 exit status: 0
Job 2 completed
Job 2 exit status: 0
Job 3 exit status: 0

=== Stability Test ===
All jobs completed successfully

=== Failure Detection Test ===
Failure detected successfully

=== Performance Analysis ===
CPU execution time: 0.0031 seconds

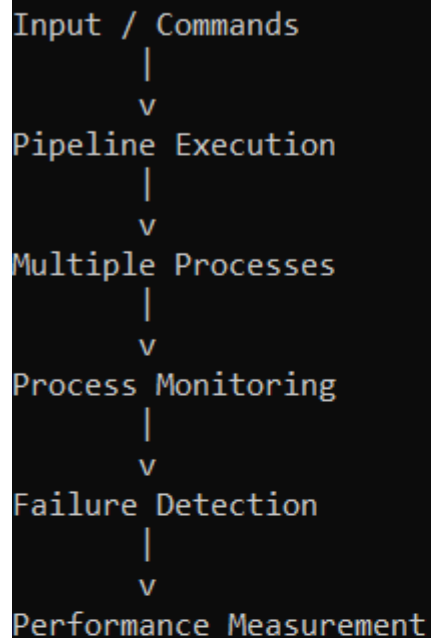
```

nano documentation.txt:

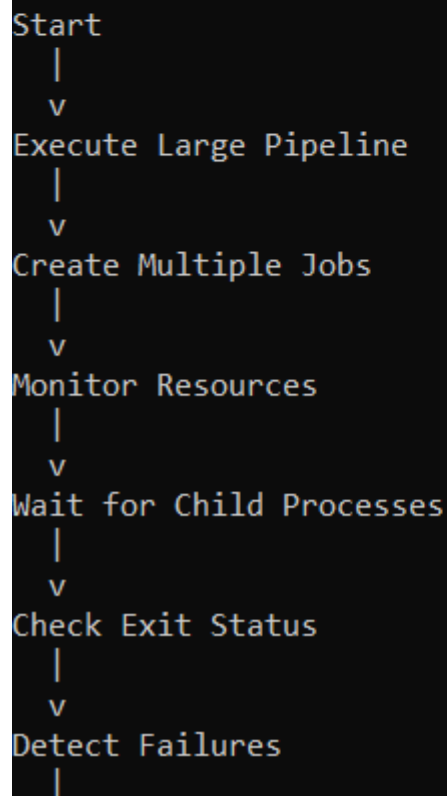
TASK 23 DOCUMENTATION

1. Architecture

The program contains these main parts:



2. Flow Diagram



```

Detect Failures
|
v
Measure Performance
|
v
End

```

3. Execution Path

The program first executes a pipeline containing multiple connected commands.

Next, three child processes are created using `fork()`.

Each child performs a small task and terminates.

The parent monitors the processes and waits for their completion using `waitpid()`.

The program then checks exit status and performs a failure test.

Finally, execution time is measured.

4. System Calls Used

`fork()`
Creates child processes.

`waitpid()`
Waits for child process completion.

`getpid()`
Retrieves the process ID.

`system()`
Executes shell commands and pipelines.

`exit()`
Terminates child processes.

5. Design Decisions

Multiple child processes are used to test concurrent execution.

A long pipeline is used to test command flow.

Exit status is checked to detect failures.

Resource usage is displayed using ps.

Execution time is recorded for performance analysis.

6. Review

The program successfully demonstrates:

- Large pipeline execution
- Multiple job execution
- Stability testing
- Resource monitoring
- Failure detection
- Performance analysis
- Process synchronization

```
nishanth@LAPTOP-6Q3ON07H:~/task23$ nano documentation.txt
nishanth@LAPTOP-6Q3ON07H:~/task23$ cat documentation.txt
TASK 23 DOCUMENTATION
```

1. Architecture

The program contains these main parts:

```
Input / Commands
    |
    v
Pipeline Execution
    |
    v
Multiple Processes
    |
    v
Process Monitoring
    |
    v
Failure Detection
    |
    v
Performance Measurement
```

2. Flow Diagram

```
Start
  |
  v
Execute Large Pipeline
  |
  v
Create Multiple Jobs
  |
  v
Monitor Resources
  |
```

```
Execute Large Pipeline
|
v
Create Multiple Jobs
|
v
Monitor Resources
|
v
Wait for Child Processes
|
v
Check Exit Status
|
v
Detect Failures
|
v
Measure Performance
|
v
End
```

3. Execution Path

The program first executes a pipeline containing multiple connected commands.

Next, three child processes are created using `fork()`.

Each child performs a small task and terminates.

The parent monitors the processes and waits for their completion using `waitpid()`.

The program then checks exit status and performs a failure test.

Finally, execution time is measured.

4. System Calls Used

`fork()`
Creates child processes.

`waitpid()`
Waits for child process completion.

4. System Calls Used

`fork()`

Creates child processes.

`waitpid()`

Waits for child process completion.

`getpid()`

Retrieves the process ID.

`system()`

Executes shell commands and pipelines.

`exit()`

Terminates child processes.

5. Design Decisions

Multiple child processes are used to test concurrent execution.

A long pipeline is used to test command flow.

Exit status is checked to detect failures.

Resource usage is displayed using `ps`.

Execution time is recorded for performance analysis.

6. Review

The program successfully demonstrates:

- Large pipeline execution
- Multiple job execution
- Stability testing
- Resource monitoring
- Failure detection
- Performance analysis
- Process synchronization

